Introduction to Parallel Computing 3

Parallel Programming Models and Interfaces

Designing Parallel Algorithms

- First understand the problem that you wish to solve in parallel
  - Establish the baseline and measure its performance

- Identify hotspots where most of the work is done
  - Profile and analyze performance
  - Focus on parallelizing the hotspots
  - Maximize parallelism, minimize overheads and load imbalance

- Identify bottlenecks that limit parallel performance
  - Restructure the algorithm to reduce/eliminate these bottlenecks

- Evaluate end-to-end performance across different input sizes and degrees of parallelism

- Use optimized third-party parallel software and math libraries when available



Task Decomposition

◎ **Analyze task and data dependences of your algorithm**

- Dependences constrain execution order and limit available parallelism
- Restructure the computation while preserving correctness
- Give higher priority to more critical jobs

◎ **Choose task sizes that balance parallelism and execution overhead**

- Make tasks large enough to amortize scheduling and coordination overhead
- Provide enough tasks to utilize the available compute resources
- Larger tasks often improve locality and reduce overhead, but can increase load imbalance
- When computation and communication costs are difficult to predict, consider combining static decomposition with adaptive runtime scheduling

Scheduling and Communication

- **Avoid shared updates as much as possible**
  - Use private or distributed synchronization/communication primitives
- **Exploit data locality and task affinity**
  - Reduces the # of communication required
  - Keep related work and reusable data close to the processing units that use them
- **Overlap communication with computation**
  - To hide communication delay
- *In some cases, recompute instead of communicate*
  - Lose computation performance to gain communication performance
- *In some cases, it may be okay to use "stale" data*
  - Especially, for iterative algorithms that will eventually converge no matter what
  - Or problems that can tolerate approximate solutions

Parallel Programming Interfaces

Parallel Programming Models

Interface	Execution unit	Data organization	Programmer responsibility
Pthreads	CPU threads	Shared address space	Create threads, divide work, synchronize
OpenMP	Threads and tasks	Shared host data; target mapping	Mark parallel work and data attributes
CUDA (OpenCL, HIP)	GPU threads and blocks	Host/device memory spaces	Map work to threads and manage device data
OpenMP target / OpenACC	Offloaded loops	Host/device data environments	Specify parallel loops and data movement
MPI	Processes (ranks)	Process-local memory	Distribute data and communicate

Parallel Programming Models

Level of abstractions

Interface	Execution unit	Data organization	Programmer responsibility
Pthreads	CPU threads	Shared address space	Create threads, divide work, synchronize
OpenMP	Threads and tasks	Shared host data; target mapping	Mark parallel work and data attributes
CUDA (OpenCL, HIP)	GPU threads and blocks	Host/device memory spaces	Map work to threads and manage device data
OpenMP target / OpenACC	Offloaded loops	Host/device data environments	Specify parallel loops and data movement
MPI	Processes (ranks)	Process-local memory	Distribute data and communicate

Parallel Programming Models

Supported parallel hardware scope

Interface	Execution unit	Data organization	Programmer responsibility
Pthreads	CPU threads	Shared address space	Create threads, divide work, synchronize
OpenMP	Threads and tasks	Shared host data; target mapping	Mark parallel work and data attributes
CUDA (OpenCL, HIP)	GPU threads and blocks	Host/device memory spaces	Map work to threads and manage device data
OpenMP target / OpenACC	Offloaded loops	Host/device data environments	Specify parallel loops and data movement
MPI	Processes (ranks)	Process-local memory	Distribute data and communicate

Running Example: Matrix-Vector Multiplication

- Row-major storage
- Inputs A and x are read-only
- Each row produces one output element
- Different rows are independent

$y = A\,x$ A: $m \times n$
x: n elements y: m elements

```
void matvec(int m, int n, const double *A,
            const double *x, double *y) {
    for (int i = 0; i < m; ++i) {
        double sum = 0.0;
        for (int j = 0; j < n; ++j)
            sum += A[i*n + j] * x[j];
        y[i] = sum;
    }
}
```

Shared Memory: Pthreads

◎ POSIX threads is standard threading implementation for UNIX-like operating systems (Linux, MAC OS X)

- Threads in one process shared global and heap data
- Each thread has its own stack
- Knowledge of pthreads can easily be transferred to programming other widely used threading specifications (e.g., Java threads)

◎ Programmers directly manage work and data distribution

- Create threads, pass arguments, assign work, and join completed threads

Matrix-Vector Multiplication: pthreads

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int m, n, thread_count;
void *Pth_mat_vect(void *rank);                 /* thread function */
int main(int argc, char *argv[]) {
        long thread;
        pthread_t * thread_handles;
        thread_count = strtol(argv[1], NULL, 10); /* Get number of threads */
        thread_handles = malloc(thread_count*sizeof(pthread_t));
        for (thread=0;thread<thread_count;thread++)
                pthread_create(&thread_handles[thread], NULL, Pth_mat_vect, (void*) thread);
        /* Join the result from each thread */
        for (thread=0;thread<thread_count;thread++)
                pthread_join(thread_handles[thread], NULL);
        free(thread_handles);
        return 0;
} /* main */
```

Each thread receives a distinct thread id that remain valid until join

Matrix-Vector Multiplication: Pthreads

```
void *Pth_mat_vect(void *rank) {
        long my_rank = (long) rank;
        int i,j;
        int my_first_row = my_rank * m / thread_count;
        int my_last_row = (my_rank + 1) * m / thread_count;

        for (i=my_first_row;i<my_last_row;i++) {
                y[i] = 0.0;
                for (j=0;j<n;j++)
                        y[i] += A[i][j] * x[j];
        }
        return NULL;
}
```

Example: m = 10, thread_count = 3

Thread 0: rows [0, 3) Thread 1: [3, 6) Thread 2: [6, 10)

Shared Memory: OpenMP

- A directive-based programming model for C/C++ and Fortran
  - The compiler and runtime implement parallel regions, worksharing, tasks, and SIMD

- Less explicit thread-management code provides higher programmability and portability
  - Users only have to express parallelism through directives

- Support host parallelism and accelerator offloading
  - Target directives extend the API to devices
  - OpenMP 6.0 is the current released version (device offloading began in 4.0)

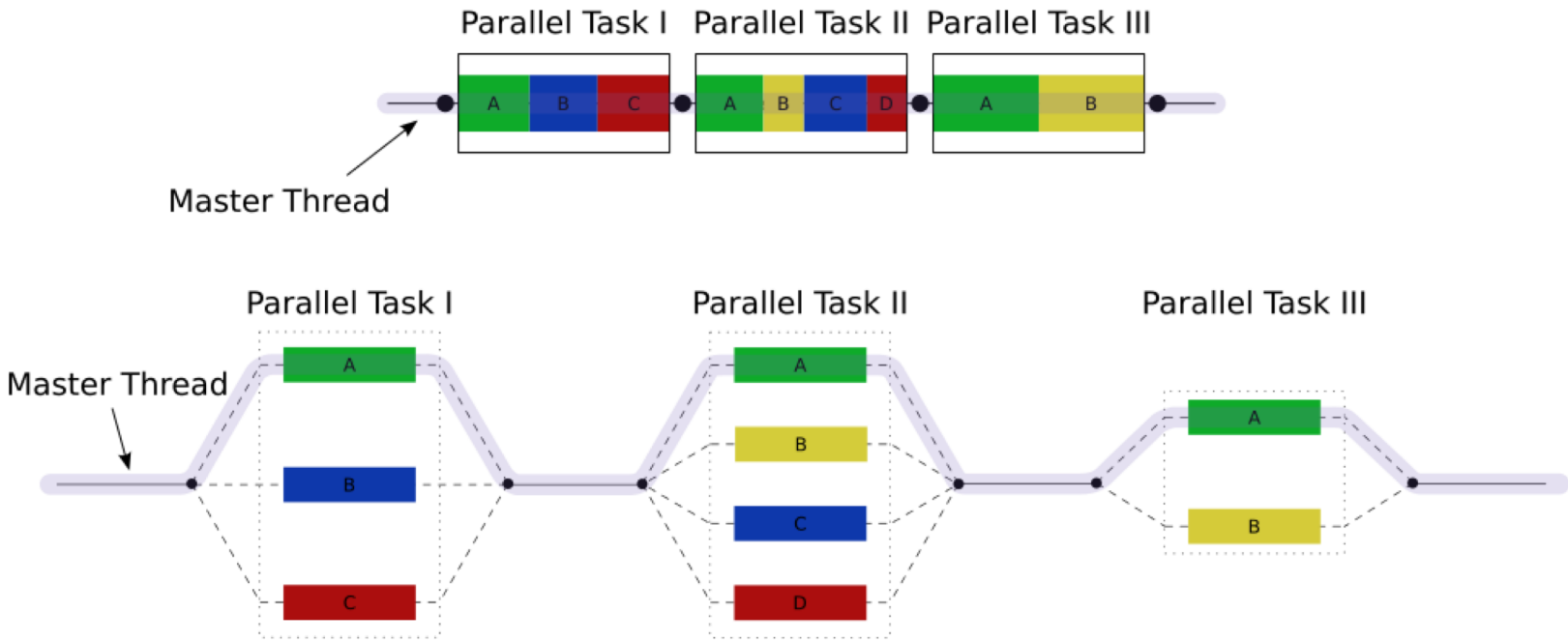
OpenMP Building Blocks

Purpose	C/C++ constructs or controls
Create parallel work	parallel, task
Distribute work	for, sections, single
Control shared access and ordering	barrier, critical, atomic, taskwait
Specify data attributes	shared, private, firstprivate, reduction
Control the runtime	omp_set_num_threads(), omp_get_num_threads()
Set the environment	OMP_NUM_THREADS

Fork-Join Model

◎ Fork: The master thread creates a team of threads which are executed in parallel

◎ Join: when the team members complete the work, the synchronize and terminate. The master thread continues sequentially



Matrix-Vector Multiplication: OpenMP C

```
Void Omp_mat_vect(int m, int n, double * restrict y,
                  double * restrict A, double * restrict x) {
        int i, j;

#pragma omp parallel for default (none) \
                  shared(m, n, A, x, y) private (i, j)
        for (i = 0; i < m; i++) {
                  a[i] = 0.0;
                  for (j = 0; j < n; j++)
                           y[i] += A[i*n+j] * x[j];
        } /* End of omp parallel for */
```

- The runtime assigns rows to threads
- A, x, and y are shared, but each y[i] has one writer

Message-Passing: MPI

- International standard
  - MPI 1.0 released in 1994
  - Current version is 5.0 (2025)
  - Interfaces in C/C++ and Fortran

- MPI processes assume separate address spaces
  - Processes exchange data explicitly (point-to-point messages and collective communication)

- The same parallel programming model works within and across nodes
  - Works on both distributed memory and shared memory hardware
  - Several ranks can share one physical node

MPI Basics

- Two common MPI functions
  - MPI_SEND(): send a message
  - MPI_RECV(): receive a message
  - Functions like write and read statements
  - Both are blocking operations
- Non-blocking operations
  - MPI_ISEND()
  - MPI_IRECV()
- The user can check for data at a later stage in the program without waiting
  - MPI_WAIT()
  - MPI_TEST()
  - Possible to overlap communication and computation

MPI Program Template

```
#include “mpi.h”

int main(int argc, char **argv) {
/* Initialize MPI */
MPI_Init(&argc, &argv);

/* Allow each processor to determine its role */
int num_processor, my_rank;
MPI_Comm_size(MPI_COMM_WORLD, &num_processor);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

/* Send and receive some stuff here */
...
/* Finalize */
MPI_Finalize();
}
```

Each process runs its own copy of main and has its own local variables

Matrix-Vector Multiplication: MPI

```
MPI_Bcast(x, n, MPI_DOUBLE, 0, comm);
MPI_Scatter(A, rows*n, MPI_DOUBLE,
            local_A, rows*n, MPI_DOUBLE, 0, comm);

for (int i = 0; i < rows; ++i) {
    double sum = 0.0;
    for (int j = 0; j < n; ++j)
        sum += local_A[i*n + j] * x[j];
    local_y[i] = sum;
}

MPI_Gather(local_y, rows, MPI_DOUBLE,
           y, rows, MPI_DOUBLE, 0, comm);
```

◎ Assume rows = m / size, m % size == 0, comm = MPI_COMM_WORLD

Collective Communication

Collective	Data operation	Example use
Bcast	One value or array to all ranks	Replicate the input vector x
Scatter / Gather	Distribute / collect separate pieces	Partition A / assemble y
Allgather	Collect pieces at every rank	Replicate a partitioned vector
Reduce / Allreduce	Combine values at one / all ranks	Global minimum or gradient sum
Alltoall	Every rank sends a distinct piece to every rank	Redistribution or transpose

MPI Synchronization

- Implicit synchronization
  - Blocking communication
  - Collective communication

- Explicit synchronization
  - MPI_Wait/MPI_Test
    - Completes or test requests (do not synchronize all ranks)
  - MPI_Barrier
    - Returns only after every rank has entered the call

Heterogeneous Parallelism

- Low-level programming
  - OpenCL, SYCL: portable
  - CUDA, HIP: proprietary

- High-level programming
  - OpenMP target v4.0
  - OpenACC
  - Allows for a single code base that may be compiled on both multicore and GPUs

Matrix Multiplication: CUDA

```
__global__ void matvec_gpu(int m, int n, const double *A,
                           const double *x, double *y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= m) return;
    double sum = 0.0;
    for (int j = 0; j < n; ++j)
        sum += A[i*n + j] * x[j];
    y[i] = sum;
}

// Host: d_A and d_x already hold the input data
matvec_gpu<<<(m + 255) / 256, 256>>>(m, n, d_A, d_x, d_y);
```

- One GPU thread computes one row
- `(blockIdx.x * blockDim.x + threadIdx.x)` identifies executing thread

Directive-Based Accelerator Offloading

- Host-centric: host device offloads code regions and data to accelerators
  - The compiler generates device code from the annotated regions

- Model the device as a distinct execution and data environment
  - The device executes offloaded work using its own parallel resources
  - Need to control data movement between host and device
  - Host and device may have separate or physically shared memory

- Mapping clause: defines the relationship between memory on the host device and memory on the accelerator
  - OpenMP uses map clauses, while OpenACC uses copyin, copyout, and create clauses
  - Can keep reused data on the device across kernels

Matrix-Vector Multiplication: OpenMP and OpenACC

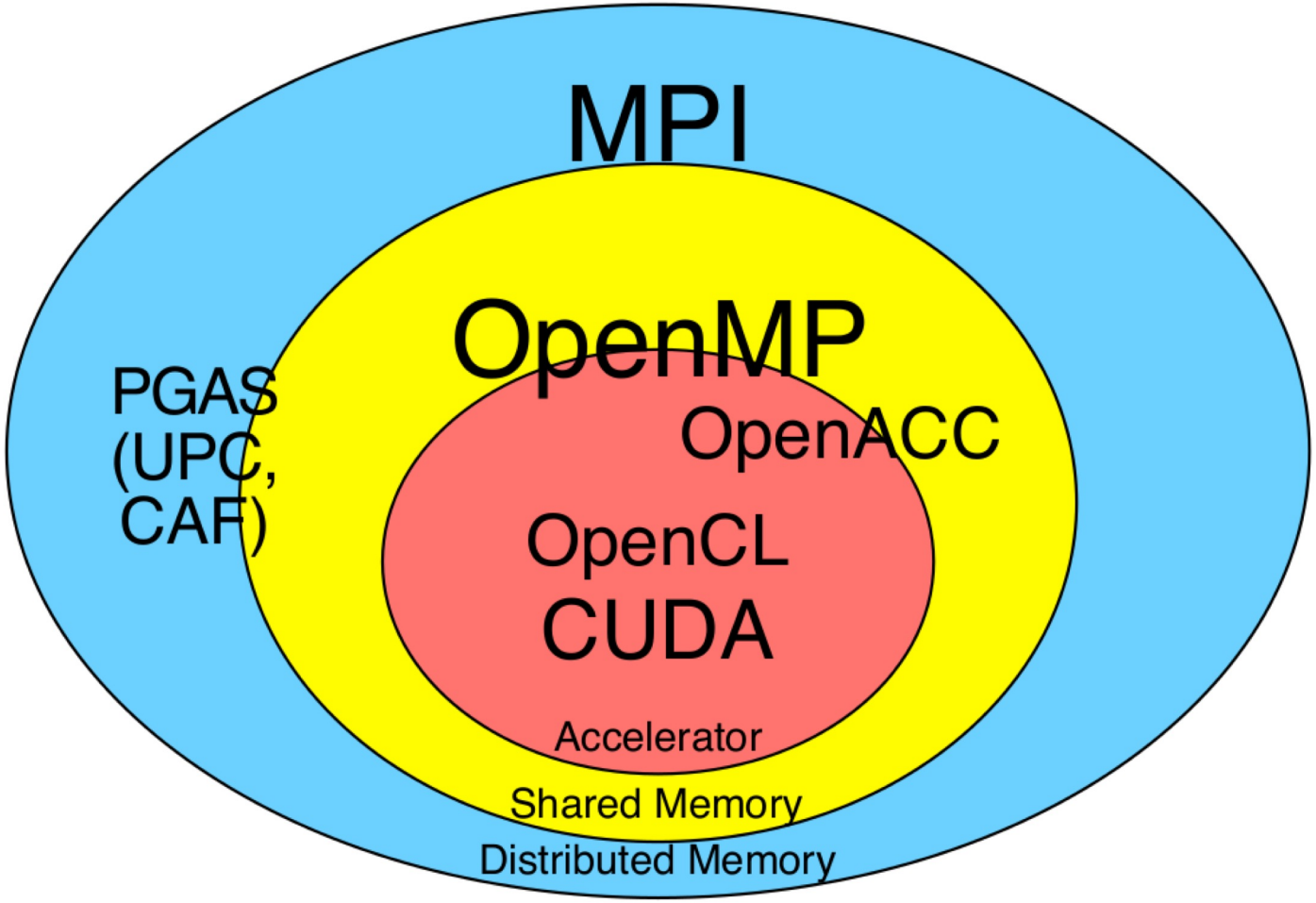
```
void matvec_target(int m, int n, const double *A,
                   const double *x, double *y) {
   #pragma omp target teams distribute parallel for \
      map(to: A[0:m*n], x[0:n]) map(from: y[0:m])
   for (int i = 0; i < m; ++i) {
      double sum = 0.0;
      for (int j = 0; j < n; ++j)
         sum += A[i*n + j] * x[j];
      y[i] = sum;
   }
}
```

```
void matvec_acc(int m, int n, const double *A,
                const double *x, double *y) {
   #pragma acc parallel loop \
      copyin(A[0:m*n], x[0:n]) copyout(y[0:m])
   for (int i = 0; i < m; ++i) {
      double sum = 0.0;
      #pragma acc loop seq
      for (int j = 0; j < n; ++j)
         sum += A[i*n + j] * x[j];
      y[i] = sum;
   }
}
```

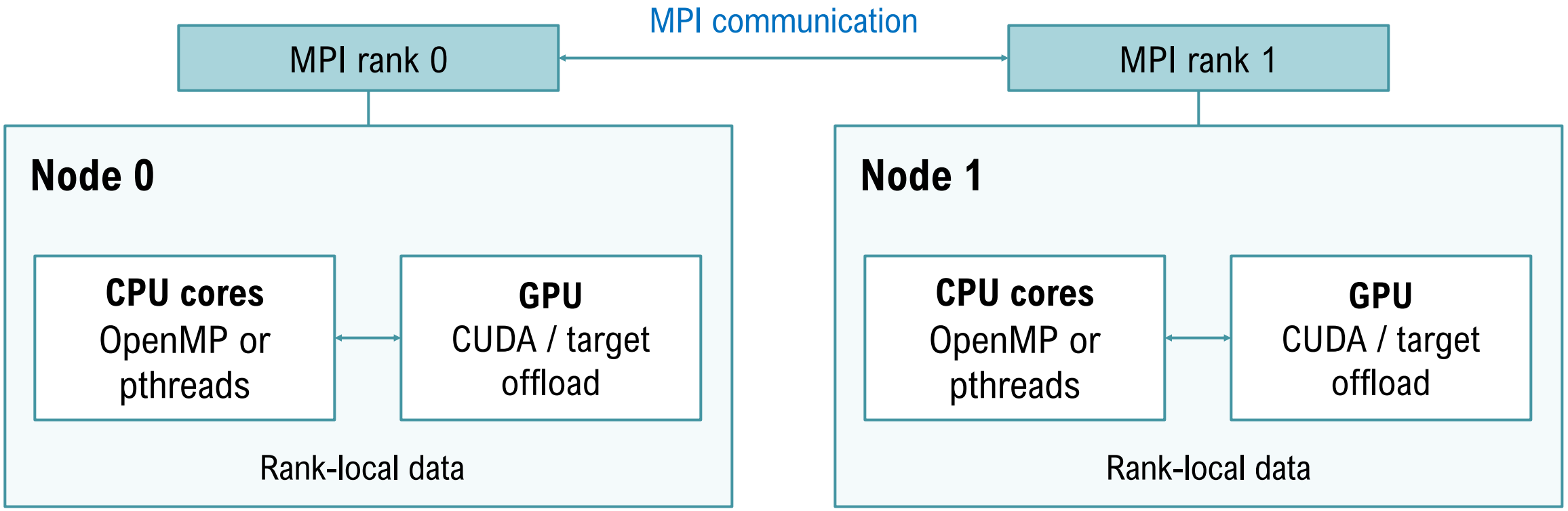
◎ The directive before i loop exposes independent rows and executes them in parallel on a device like GPUs

◎ For OpenACC, you can also use `#pragma acc kernels` directive to implicitly direct the compiler to analyze and decide what to parallelize

Hybrid Computing: MPI+X



Hybrid Computing: MPI+X



◎ Combine process-level communication with parallel execution within each node

Constantly Evolving…

- Need to consider both algorithms and architectures as well as their interplay
  - Both are rapidly expanding
- For example,
  - Huge data sets
  - Unstructured data sets
  - Heterogeneity in hardware and software
  - Performance portability across a wide range of systems
  - Need for balancing speed, power, and cost
- The importance of parallel programming interfaces every growing
  - Portability, flexibility, performance

Acknowledgments

◎ Slides are adapted from and inspired by

- Heterogeneous parallel computing at POSTECH, Multicore programming primer at MIT, Parallel computer architecture and programming at CMU, Parallel architecture at UIUC, Introduction to parallel computing at U of Chicago

Next Class

Chip Multiprocessors and Vector Computers